Comparable vs Comparator

Comparable

- **Package**: `java.lang`
- **Interface**: `Comparable<T>`
- **Method**: `int compareTo(T o)`
- **Purpose**: Defines the *natural ordering* of objects.
- **When to use**: If you want a class to have a **default sorting logic**.
- **Limitation**: A class can only implement **one** `compareTo` method, so it can have only one natural order.

Example

```java
class User implements Comparable<User> {
    private int id;
    private String name;

    @Override
    public int compareTo(User other) {
        return Integer.compare(this.id, other.id); // sort by id
    }
}

```

Usage:

```java
List<User> users = ...
Collections.sort(users); // uses compareTo()

```

Comparator

- **Package**: `java.util`
- **Interface**: `Comparator<T>`
- **Method**: `int compare(T o1, T o2)`
- **Purpose**: Defines a **custom ordering** (can have multiple strategies).
- **When to use**: If you want **different sorting logics** for the same class.
- **Flexibility**: You can create as many comparators as you like.

Example

```java
class UserNameComparator implements Comparator<User> {
    @Override
    public int compare(User u1, User u2) {
        return u1.getName().compareTo(u2.getName()); // sort by name
    }
```

```
}

```

Usage:

```
List<User> users = ...
Collections.sort(users, new UserNameComparator()); // uses compare()

```

Key Differences Table

Aspect	Comparable	Comparator
Package	`java.lang`	`java.util`
Method	`compareTo(T o)`	`compare(T o1, T o2)`
Ordering	Natural (default)	Custom (multiple possible)
Implementation	Implemented by the class itself	Implemented as separate class/lambda
Number allowed	Only one per class	Multiple comparators per class

---

> **Rule of thumb**:
>
> - Use **Comparable** when there is **one natural order** (e.g., numbers, dates, alphabetical).
> - Use **Comparator** when you need **multiple sorting strategies** (e.g., sort Users by age, then by name).

Question:
What happens if:

```
Collections.sort(list);
```

is called on a list of objects that **do not implement Comparable** and **no Comparator is provided**?

If:

- Objects do **NOT implement Comparable**

- AND no Comparator is provided

Runtime exception

```
java.lang.ClassCastException
```

There is **NO default sorting**.

TASK 1 — Comparable

You must **complete the code**, not rewrite it.

```java
// File: Person.java
public class Person implements Comparable<Person> {

    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // TODO: implement comparison logic
}

```

Requirement:

Sort persons by **age ascending**.

If age is equal → sort by **name alphabetically**.

TASK 2 — Comparator

```java
// File: PersonNameDescComparator.java
import java.util.Comparator;

public class PersonNameDescComparator implements Comparator<Person> {

    @Override
    public int compare(Person p1, Person p2) {
        // TODO
    }
}

```

Requirement:

Sort by **name descending**, ignore case.

Correct implementation

```java
@Override
public int compareTo(Person other) {
    int ageCompare = Integer.compare(this.age, other.age);
    if (ageCompare != 0) {
        return ageCompare;
    }
    return this.name.compareToIgnoreCase(other.name);
```

```
}

```

```java
import java.util.Comparator;

public class PersonNameDescComparator implements Comparator<Person> {
@Override
public int compare(Person p1, Person p2) {
    return p2.getName().compareToIgnoreCase(p1.getName());
}
```